



Université Hassan Premier :
Faculté des sciences et techniques
De Settat



Département de Mathématiques et Informatique

**Simulateur de
microprocesseur
Motorola 6809**

Réalisé par :

Ahmed elkettani

saad maerof

amor louafi

Encadré par :

:professeur Hicham benalla

Année universitaire : 2025/2026

Instructions Implémentées

Notre simulateur prend en charge 23 instructions du jeu d'instructions complet du 6809, soigneusement sélectionnées pour couvrir les principales catégories fonctionnelles :

1. Transfert mémoire

Charge une valeur dans le registre A :LDA

Charge une valeur dans le registre B :LDB

Stocke le contenu de A en mémoire :STA

Stocke le contenu de B en mémoire :STB

Exemple de code d'assemblage :

```
if (mnemonic.equals("LDA")) {  
    if (operande.startsWith("#")) {  
        String vraioperande = operande.substring(2);  
        int op = Integer.parseInt(vraioperande, 16);  
        return new byte[] {(byte)0x86, (byte)op};  
    }  
    else if (operande.startsWith("<")){  
        String vraioperande = operande.substring(2);  
        int op = Integer.parseInt(vraioperande, 16);  
        return new byte[] {(byte)0x96, (byte)op};  
    }  
}
```

2. Opérations arithmétiques

Addition avec le registre A :ADDA

Addition avec le registre B :ADDB

Soustraction avec A :SUBA

Soustraction avec B :SUBB

Exemple d'exécution :

```
else if (instruction == (byte)0x8B) {  
    byte valeur = memoire[pc + 1];  
    int resultat = registres.getA() + (valeur & 0xFF);  
    registres.setA(resultat & 0xFF);  
}
```

```
registres.setPC(pc + 2);  
}
```

3. Opérations logiques

ET logique avec A :AND A

OU logique avec A :OR A

4. Incrémentation/Décrémentation

Incrémente/Décrémente A :INCA/DECA

Complément et incrémentation :COMA /INC

5. Transfert entre registres

Transfert entre registres ($A \leftrightarrow B, X \leftrightarrow Y$) :TFR

6. Sauts

Saut inconditionnel :JMP

7. Contrôle

Instruction sans opération :NOP

Interruption logicielle :SWI

8. Multiplication

Multiplication 8×8 bits :MUL

Ces instructions sont implémentées avec plusieurs modes d'adressage : immédiat, direct, étendu, indexé, relatif et indirect.

```
public Registres(int PC, int A, int B, int X, int Y, int S, int U, int DP, int CCR) {  
}  
}
```

2. Classe Conversioninstruc

Cette classe implémente l'assembleur intégré qui convertit les mnémoniques assembleur en code machine :

```
public class Conversioninstruc {  
    public static boolean b;  
    public byte[] convertir(String mnemonic, String operande) {  
    }  
}
```

Fonctionnalités clés :

Prise en charge des préfixes d'adressage : # (immédiat), < (direct), > (étendu)

Gestion des modes indexés : ,x, ,x+, ,-x

Support du mode indirect : [adresse]

Conversion des adresses hexadécimales en bytes

3. Classe Motorola

Cœur du simulateur qui exécute les instructions :

```
public class Motorola {  
    Registres registres = new Registres(0,0,0,0,0,0,0,0,0);  
    byte[] memoire = new byte[0xFFFF];  
    private int prochaineAdresse = 0xC000;  
  
    public void chargerEnMemoire(byte[] programme) {  
    }  
  
    public void executerInstruction() {  
    }  
  
    private int tailleInstructionIndexee(byte postbyte) {  
    }  
  
    private int calculerAdresseIndexee(byte postbyte, int pc) {  
    }  
}
```

Cycle d'exécution :

1. Lecture de l'opcode à l'adresse PC
2. Décodage de l'instruction
3. Calcul de l'adresse effective (si nécessaire)
4. Exécution de l'opération
5. Mise à jour du PC et des flags CCR
6. Classe Interfaceprojetmotorola

Interface graphique complète développée avec Swing :

```
public class Interfaceprojetmotorola extends JFrame {  
    private JTextField pcField, aField, bField, xField, yField;  
    private JTextArea ramArea, romArea, editeurArea;  
    private JCheckBox[] ccrCheckboxes;
```

```
public Interfaceprojetmotorola() {  
    setTitle("Simulateur Motorola 6809");  
    setSize(900, 600);  
}
```

```
private void ouvrirEditeur() {  
}
```

```
private void executerPasAPas() {  
}
```

```
private void mettreAJourAffichage() {  
}  
}
```

Interface Graphique

L'interface graphique a été conçue avec Java Swing pour offrir une expérience utilisateur complète et pédagogique :

Structure de l'interface :

1. Barre de menu : Accès rapide aux fonctionnalités
Bouton Éditeur pour saisir du code
Menu Mémoire pour visualiser différentes zones
2. Panel des registres (à gauche) :
Affichage hexadécimal de tous les registres
Séparation claire entre registres 8 et 16 bits
Groupement des accumulateurs A et B sous le registre D
3. CCR (Condition Code Register) :
Visualisation sous forme de 8 cases à cocher
Chaque flag (E, F, H, I, N, Z, V, C) représenté individuellement
Mise à jour automatique après chaque instruction
4. Mémoire RAM/ROM (à droite) :
Affichage des premières adresses de chaque zone
Format adresse/valeur en hexadécimal
Police monospace pour un alignement parfait

5. Éditeur intégré :

```
private void ouvrirEditeur() {  
    JDialog dialog = new JDialog(this, "Éditeur d'instructions", true);  
    dialog.setSize(600, 500);  
    editeurArea = new JTextArea(20, 50);  
    editeurArea.setFont(new Font("Monospaced", Font.PLAIN, 12));  
}
```

Permet de saisir plusieurs instructions (une par ligne)

Boutons pour exécuter en une fois ou pas à pas

Validation syntaxique basique

Fonctionnalités avancées :

Exécution pas à pas :

```
private void preparerPasAPas() {  
    instructions = editeurArea.getText().split("\n");  
    indexInstruction = 0;  
    executerPasAPasAvecDialogue();  
}
```

Confirmation après chaque instruction

Affichage du numéro d'instruction en cours

Visualisation mémoire détaillée :

```
private void afficherZoneMemoire(String titre, int adresseDebut, int adresseFin) {  
}
```

Réinitialisation complète :

```
private void reinitialiserEtat() {  
    registres = new Registres(0x0000, 0x00, 0x00, 0x0000, 0x0000, 0x0000, 0x0000, 0x00,  
    0x00);  
    motorola = new Motorola();  
    motorola.registres = registres;  
    mettreAJourAffichage();  
}
```

Exemple et Démonstration Pratique

Scénario d'utilisation complet :

1. Saisie du programme dans l'éditeur :

```
LDA #$15
ADDA #$20
STA $1000
LDB #$30
ADDB A
```

2. Exécution pas à pas :

Après LDA #\$15 : A = 0x15, PC avance de 2 octets
Après ADDA #\$20 : A = 0x35, flag Z=0, N=0
Après STA \$1000 : Mémoire[0x1000] = 0x35
Visualisation en temps réel dans l'interface

3. Inspection de la mémoire :

Accès via le menu "Mémoire → Afficher toute la mémoire"

Affichage formaté :

Adresse Valeur

1000 35

1001 00

4. Visualisation des registres :

Registre D affiche A et B côte à côte

CCR montre les flags mis à jour

PC indique l'adresse de la prochaine instruction

Cas de test avancé :

```
TFR A,B
```

```
LDA <$10
```

```
ADDA ,x
```

```
STA [$2000]
```

```
JMP $C000
```

Défis Techniques et Solutions

1. Gestion des modes d'adressage

Problème : Le 6809 supporte de nombreux modes d'adressage complexes.

Solution : Implémentation d'une méthode de calcul d'adresse générique :

```
private int calculerAdresseIndexee(byte postbyte, int pc) {
```

```

int post = postbyte & 0xFF;
if (post == 0x00) {
    return registres.getX();
}
else if (post == 0x80) {
    int adresse = registres.getX();
    registres.setX(adresse + 1);
    return adresse;
}
}

```

2. Synchronisation interface/simulateur

Problème : Maintenir l'interface à jour avec l'état du simulateur.

Solution : Méthode mettreAJourAffichage() appelée systématiquement :

```

private void mettreAJourAffichage() {
    pcField.setText(String.format("%04X", registres.getPC()));
    aField.setText(String.format("%02X", registres.getA()));
    int ccr = registres.getCCR();
    eCheckBox.setSelected((ccr & 0x80) != 0);
}

```

3. Gestion de la mémoire

Problème : Simuler 64 Ko de mémoire efficacement.

Solution : Tableau de bytes avec séparation logique RAM/ROM :

```
byte[] memoire = new byte[0xFFFF];
```

Résultats et Validation

Le simulateur a été validé sur plusieurs critères :

1. Exactitude fonctionnelle :

Instructions de base (LDA, STA) fonctionnent correctement

Modes d'adressage validés individuellement

CCR mis à jour conformément à la documentation

2. Performance :

Exécution instantanée des instructions

Interface réactive même avec des programmes longs
Gestion mémoire efficace

3. Utilisabilité :

Interface intuitive pour les débutants
Documentation intégrée via les messages d'aide
Gestion des erreurs robuste

Perspectives d'Amélioration

Bien que fonctionnel, le simulateur pourrait être enrichi :

1. Extensions techniques :

Ajout des instructions manquantes du 6809
Support des interruptions matérielles
Émulation des périphériques d'E/S

2. Améliorations interface :

Éditeur avec coloration syntaxique
Points d'arrêt et watchpoints
Export/import de programmes binaires

3. Fonctionnalités pédagogiques :

Mode tutoriel avec explications pas à pas
Visualisation du pipeline d'exécution
Comparaison avec d'autres architectures

Conclusion

Ce projet a permis de développer un simulateur complet et fonctionnel du microprocesseur Motorola 6809, offrant à la fois une précision technique et une approche pédagogique. L'architecture modulaire en Java, combinée à une interface graphique intuitive, en fait un outil valable pour l'enseignement de l'architecture des microprocesseurs.

Les principaux accomplissements sont :

Une implémentation fidèle du jeu d'instructions avec multiples modes d'adressage
Une interface graphique complète permettant visualisation et interaction
Une architecture extensible facilitant l'ajout de nouvelles fonctionnalités
Une documentation intégrée guidant l'utilisateur

Ce simulateur constitue une base solide pour l'expérimentation et l'apprentissage, démontrant comment les concepts théoriques d'architecture se concrétisent dans un environnement logiciel contrôlé.